Search Engine

Stage 3

María Alonso León, Víctor Gil Bernal, Kimberly Casimiro Torres

Big Data
School of Computer Engineering (EII)
Las Palmas de Gran Canaria University, Spain

January 2025

Contents

1 Introduction **2**

2 Problem Statement **2**

3 Methodology **3**
3.1 SOLID Design Principles and Design Patterns . . . 4
3.1.1 Single Responsibility Principle (SRP) . . . 4
3.1.2 Open/Closed Principle (OCP) . . . 4
3.1.3 Liskov Substitution Principle (LSP) . . . 4
3.1.4 Interface Segregation Principle (ISP) . . . 4
3.1.5 Dependency Inversion Principle (DIP) . . . 4
3.2 Architecture Structure . . . 5
3.2.1 Adapters: . . . 5
3.2.2 Application: . . . 7
3.2.3 Config: . . . 8
3.2.4 Domain: . . . 8
3.2.5 Main: . . . 9
3.2.6 Utils: . . . 10
3.3 API (Application Programming Interface): . . . 10
3.4 UI (User Interface): . . . 10

4 Experiments and Tests **11**

5 Performance Comparison of Search Implementations **18**
5.1 Stage 1 (Python) and Stage 2 (Java) . . . 19
5.2 Stage 2 (Java) and Stage 3 (Java) . . . 20

6 System Performance Analysis **22**
6.1 How many concurrent clients can the system handle? . . . 22
6.2 Where are the bottlenecks identified? . . . 22
6.3 Key Findings for Scalability . . . 23

7 Conclusion **23**

8 Future Work **24**

ABSTRACT

KEYWORDS:

1. Introduction

In the current landscape of computer science, characterized by the explosion of Big Data and the growing need to efficiently manage large volumes of information, search engines have become indispensable tools. These systems not only enable quick access to critical data but also play a key role in data-driven decision-making, research, and the development of advanced applications. However, building scalable, robust, and optimized search engines remains a technical challenge, particularly when handling high volumes of data and concurrent queries.

This project, which represents the culmination of a series of previous developments, focuses on the implementation of an optimized search engine that combines technological innovation with a robust architectural design. Unlike earlier stages, where components such as the crawler, indexer, and query engine were managed as explicit modules, this approach integrates them under a unified design, maximizing system cohesion. While these functional components are still utilized, the system is no longer divided into conventional modules but is instead organized around a flexible and cohesive structure based on Hexagonal Architecture. This architectural approach ensures the separation between core business logic and technical implementations, enabling the system to be adaptable and resilient to technological changes.

A key feature of this project is the use of Docker and its integration with Hazelcast to optimize distributed processing and data management. Docker allows for the creation of lightweight containers that encapsulate instances of the search engine, facilitating deployment across multiple nodes. These nodes, distributed via Docker, interact with Hazelcast to handle shared data and coordinate in-memory storage operations. Hazelcast, as an in-memory computing platform, enables nodes to quickly access indexes and metadata without the overhead of frequent disk access, significantly improving response times in an environment with concurrent queries.

Regarding the data structures used to enhance performance, new data structures such as Kryo, Avro and Binary Java Serialization. These tools optimize the serialization and deserialization of data, facilitating more efficient handling of inverted indexes and metadata. This combination of technologies reduces latency in data processing and ensures greater flexibility in managing various types of data within the system.

The implemented design also addresses some of the most pressing challenges in modern search engines, such as efficiently handling concurrent queries and scalability. In this project, the search engine is deployed across multiple nodes using Docker containers and an Nginx-based load balancer. This approach allows for the processing of a large number of simultaneous requests while ensuring a balanced distribution of the load among nodes, improving both the efficiency and responsiveness of the system.

Additionally, this project also tackles the analysis and enhancement of system performance under high-load conditions. Through extensive stress testing and simulations with multiple concurrent clients, potential bottlenecks have been identified, ensuring that the search engine is reliable and robust in demanding scenarios.

In conclusion, this project not only consolidates the progress achieved in previous phases but also introduces an innovative design that addresses current demands for efficiency and scalability. The combined use of Hexagonal Architecture, Docker for distribution, and advanced serialization technologies position this search engine as an advanced and adaptable solution for managing large volumes of data in high-impact applications. This work represents a significant step toward the creation of next-generation search systems, ready to meet current and future challenges in the realm of Big Data.

2. Problem Statement

Search systems have been essential in enabling quick and efficient access to large volumes of information. The increasing complexity of queries and the diversity of processed data have driven the need to develop search engines that are scalable, robust, and capable of adapting to dynamic environments. Without a

well-structured technical design and an implementation that meets these demands, limitations in capacity, speed, and reliability can undermine their effectiveness.

One of the most critical problems addressed in this project is the ability of the search engine to operate in distributed environments. The deployment of multiple system instances, orchestrated through containers and supported by a load balancer, introduces challenges in coordinating instances and efficiently distributing queries. Ensuring that this design can handle a high number of concurrent client requires overcoming technical barriers related to load balancing and consistency in request processing, while also maintaining system performance under high demand.

Another key challenge is achieving seamless interoperability between the system's functional components. While the crawler, indexer, and query engine are essential, their integration within a unified architecture presents the need to redefine how they interact and optimize their operations to maximize the overall system performance. This integration must enable an efficient and uninterrupted data flow from collection to querying, effectively responding to the dynamic nature of workloads.

The design of this search engine also faces the problem of validating performance under real-world usage conditions. Load testing, aimed at simulating scenarios with a great number of simultaneous queries, is essential to identify weaknesses in the current implementation. These tests not only evaluate the system's ability to scale but also its responsiveness to sudden increases in demand, a critical aspect to ensure the engine's usability in real-world applications.

Additionally, the project seeks to address the implementation of structural improvements that support the system's future adaptability. This includes ensuring that the search engine's infrastructure can easily integrate with new technologies and maintain its relevance in the rapidly evolving data environments of today. Through this approach, the system is designed not only for current needs but also to respond effectively to future demands.

In summary, this project tackles the fundamental challenges of scalability, integration, and performance validation required to build a modern search engine capable of operating in high-complexity scenarios. The objective is to overcome the limitations of traditional systems and deliver a solution that is efficient, reliable, and adaptable, capable of meeting current and future demands in the realm of massive data.

3. Methodology

This section details the methodology used in the development of an optimized search engine designed to efficiently and robustly manage large volumes of textual data. This project represents a significant evolution from previous stages, where key areas for improvement were identified in terms of performance, scalability, and system cohesion. The methodology applied in this phase aims to address these challenges by implementing a unified architectural approach that allows the system's components to integrate seamlessly and efficiently, meeting the demands of modern applications in Big Data environments.

In contrast to initial implementations, where components such as the crawler, indexer, and query engine were managed modularly, this stage adopts a fully integrated architecture. This integration seeks to maximize system cohesion by eliminating functional barriers between components, ensuring that all parts work in a coordinated manner. This approach, aligned with the adoption of a flexible design, reduces operational complexity while guaranteeing the system's ability to scale and adapt to new demands.

The choice of Java as the primary development language reflects its capabilities to efficiently handle concurrent queries and large data volumes. Java provides a structured and controlled framework, essential for meeting the scalability and maintainability standards required in such systems. Moreover, this implementation incorporates significant improvements in data processing and storage, including the use of high-performance libraries and advanced parallelization techniques, optimizing query management and indexed data handling.

The system's design focuses on structuring it as a set of integrated components, each with a specific functionality. This design ensures that the search engine can be tested, fine-tuned, and optimized as a whole, guaranteeing its ability to respond quickly and accurately to user requests. Integrating these components within a unified architectural framework not only facilitates maintenance but also enables more agile and controlled system expansion in the future.

Additionally, extensive load testing and simulations have been conducted to evaluate the system's performance under real-world conditions. These tests have been essential for identifying potential bottlenecks and validating the search engine's ability to handle multiple simultaneous queries efficiently. The methodology includes constant iterations of design, implementation, and testing, ensuring that each phase of development aligns with the overarching goals of robustness, scalability, and efficiency.

Before detailing the specific structure of the system, this section will examine the SOLID design principles that guided the development process. These principles, along with carefully selected design patterns, provided a structured foundation that enhances the system's flexibility, robustness, and extensibility. Each principle will be analyzed in detail to demonstrate its impact on the design and functionality, establishing a solid framework for the advanced search engine required in Big Data applications.

3.1. SOLID Design Principles and Design Patterns

3.1.1 Single Responsibility Principle (SRP)

This principle states that each class or component should have a single responsibility, focusing on a specific task within the system. In the third search engine project, this principle is upheld by defining clear and specialized components within the hexagonal architecture. For example, the GutenbergCrawlerAdapter class in the external package is exclusively responsible for extracting data from the books in the Gutenberg Project, without being involved in storage or queries. Similarly, the classes in the persistence package focus solely on managing and storing the inverted index and metadata, ensuring that any changes to these functionalities do not affect other layers of the system. This approach facilitates maintenance and debugging, allowing the system to evolve without compromising its stability.

3.1.2 Open/Closed Principle (OCP)

The system is designed to be open for extension but closed for modification, meaning that new functionalities can be added without modifying existing code. This is achieved through the use of interfaces and abstract classes that allow additional search functionalities to be implemented without altering the core classes of the Query Engine. For instance, if a semantic or error-tolerant search feature is needed, new classes can be created to implement these features and connect them to the engine without changing the existing structure. This helps maintain system stability and minimizes the risk of introducing errors when adding new features.

3.1.3 Liskov Substitution Principle (LSP)

This principle ensures that derived classes can replace their base classes without affecting system functionality. In the search engine, this principle is particularly useful in implementing different types of queries. For example, the Query Engine is designed to use specific classes that implement various query types, whether through the inverted index or by querying databases like MongoDB or Neo4j. Each query type can replace others in the search engine without requiring additional modifications, allowing the system to function consistently regardless of the query's internal implementation.

3.1.4 Interface Segregation Principle (ISP)

This principle suggests that interfaces should be specific and contain only the methods necessary for each implemented functionality. In this project, the interfaces are located in the domain package and are designed precisely to address specific responsibilities. For example, interfaces such as CrawlService or IndexRepository define methods tailored to their respective tasks, avoiding unrelated functionalities. This ensures a clear separation of responsibilities and reduces dependencies between the different parts of the system. By maintaining this separation, it is guaranteed that each component can be developed, tested, and expanded independently, aligning with the project's goals of modularity and scalability.

3.1.5 Dependency Inversion Principle (DIP)

This principle states that high-level classes should not depend on low-level classes but rather on abstractions. In the search engine, this principle is applied through the use of interfaces defined in the domain package, which allow high-level modules, such as the query engine in the application package, to interact with low-level modules, such as storage adapters in persistence, without depending on their specific implementations. This approach enables low-level modules to be modified or replaced without

affecting the core logic of the system, providing greater flexibility in technology choices and ensuring that the system is prepared for future updates or changes in the data infrastructure.

Overall, the application of SOLID principles in this project contributes to a robust and well-organized architecture that facilitates maintenance, improves scalability, and allows new functionalities to be added without impacting the system's structure. These principles also ensure that each component is easily testable, which is essential for a system that must efficiently manage large volumes of textual data at scale.

3.2. Architecture Structure

In this project, we have adopted Hexagonal Architecture as the foundation for the system's design. This architecture, also known as Ports and Adapters Architecture, is an approach focused on the separation of responsibilities. It allows the system's core logic to be decoupled from technical details such as frameworks, databases, or user interfaces, resulting in a more flexible, modular, and maintainable system. Hexagonal Architecture suggests that the core logic interacts with the external world through ports (interfaces). Adapters, located at the edges of the hexagon, are responsible for translating these interactions between the external world and the internal system. This structure ensures that changes in external technologies do not directly impact the system's logic, providing greater resilience to changes. The combination of this architecture with other tools makes the system more robust and adaptable to different environments.

Hazelcast is a key tool used in this project for managing distributed in-memory data. It is an in-memory computing platform that stores and processes data distributed across a cluster of nodes, enabling fast and efficient access. In the system, Hazelcast manages inverted indexes, metadata, and the datalake in real-time, optimizing performance even in high-concurrency scenarios. Additionally, it facilitates data synchronization and replication across nodes, ensuring consistency and resilience against failures. It is also used to perform periodic cleaning tasks by synchronizing data stored in memory with persistent storage, ensuring that the system remains efficient.

On the other hand, Docker plays a critical role in the project by providing an isolated and portable execution environment for all system components. By containerizing services such as the API, UI, the search engine, and Hazelcast nodes, Docker enables rapid and reproducible deployment in any environment. This ensures that the services are scalable, easy to maintain, and consistent across different environments, from development to production. Docker's ability to manage multiple interconnected containers allows the system to fully leverage the advantages of a distributed architecture, simplifying the configuration process and ensuring that each component operates optimally and independently. The combination of Hazelcast and Docker guarantees high performance, scalability, and reliability for the system.

The following section will detail the system's structure, explaining the role of each component and its relationship with Hexagonal Architecture.

3.2.1 Adapters:

The **Adapters** package includes various adapters that manage communication with external sources and internal components. It contains specific packages for handling data in the datalake, queries, persistence, and the exposure of services through a REST API.

- **External:** It contains `GutenbergCrawlerAdapter.java`, which is responsible for extracting and transforming data from the Gutenberg Project. This adapter converts the downloaded book data into a structured format, ready to be processed by the system's internal layers.
- **Persistence:** This module implements various storage strategies to optimize data management. It includes the following:
  - **Avro:** Avro is used for the efficient serialization of structured data. In this project, its implementation enables the storage of large volumes of information in a compact format, ensuring high-speed data reading and writing. It is particularly useful for distributed systems due to its built-in schema, which ensures compatibility between applications.
  - **Binary Java Serialization:** This strategy uses Java's native serialization to transform objects into a binary format that can be easily stored and retrieved. Although it is less efficient

than other alternatives like Avro or Kryo, it offers simplicity and full compatibility within the Java ecosystem.

  - **CBOR:** The CBOR (Concise Binary Object Representation) format is used for compact and efficient serialization of binary data. In this project, it is employed to reduce the size of stored data and improve information transfer in networks with limited bandwidth.
  - **File:** The file-based storage strategy allows data to be saved in local file systems. This approach is straightforward and simple but is limited to applications that do not require high scalability or intensive concurrent access.
  - **Hazelcast:** As it was mentioned before, Hazelcast is a distributed in-memory computing platform, this datatype is the core component of our implementation, since with it several nodes (computers) can share memory, particularly for the inverted index and metadata. For that reason regardless of the data type used, the corresponding hazelcast instance will always be initialized. This enables us to hava a fast access during query operations. It is particularly useful for enhancing performance in systems that handle a high volume of concurrent requests, like this one.
  - **Kryo:** Kryo is a high-performance serialization library that excels in reducing the size of serialized data. In this project, it is used to optimize storage and data transmission efficiency, particularly in environments where performance is critical.

- **Queries:** The queries package implements different query strategies designed to address a variety of functional needs in the search system. These queries include basic, advanced, and statistical operations to efficiently analyze and retrieve information.
  - **SimpleQuery:** This class implements direct queries on the indexed data, retrieving documents that contain exact matches to the search terms provided by the user. Its implementation includes methods that interact directly with the inverted index, traversing the keys and efficiently returning the associated results. This functionality is useful when absolute precision is required, leaving no margin for errors in the input keywords.
  - **FuzzyQuery:** Designed to handle approximate queries, this class allows the retrieval of documents despite typographical errors or minor differences in the search terms. It uses algorithms such as the Levenshtein distance to calculate the degree of similarity between the query terms and the words stored in the index. Additionally, it incorporates a configurable tolerance threshold that determines how much a word can differ before being excluded as a match. This functionality is ideal for scenarios where users may input incomplete or erroneous queries.
  - **MetadataQuery:** This class enables queries based on the metadata associated with the documents. For example, a user can search for documents by filtering attributes such as the author, language, or publication year. The implementation uses optimized data structures to quickly filter results according to these criteria, interacting with a specialized repository for storing and managing metadata.
  - **WordFrequencyQuery:** This query calculates and returns the frequency of one or more words in the indexed documents. The class traverses the inverted index to count how many times each word appears in the documents, returning a detailed map that links each word to its frequency. This type of query is useful for analyzing specific trends in the indexed content.
  - **DocumentFrequencyQuery:** This query returns the number of documents containing a specific word by traversing the inverted index to calculate this value. The information obtained is frequently used in analyses where it is essential to understand the relevance of a term in relation to the document set.
  - **LanguageDistributionQuery:** This query generates a statistical distribution of the languages present in the indexed documents. It analyzes the metadata associated with each document, categorizing them by language and returning a detailed breakdown. It is particularly useful for scenarios where understanding the linguistic diversity of the processed data is essential.

- **TopAuthorsQuery:** This query retrieves an ordered list of the most frequent authors in the indexed documents. The implementation analyzes the metadata to identify and rank authors based on the number of documents associated with them, providing clear and structured results.
- **ReleaseDateRangeQuery:** This query filters documents published within a specific date range. The class uses data structures that allow quick interval-based searches, returning only documents that meet the defined temporal criterion. This is particularly relevant for historical studies or trend analyses over time.

- **REST:** The REST adapter, represented by `SearchController.java`, exposes the system's functionality through a REST API using Spark. This class handles HTTP requests from external clients, enabling operations such as text queries, retrieving statistics, and accessing metadata. Its design ensures that requests are processed efficiently and securely.

The **Application** folder contains various components for managing and executing tasks in the application. The subcomponents are:

3.2.2 Application:

The **Application** folder contains various components for managing and executing tasks in the system. Each subcomponent is detailed below:

- **clean:** This subcomponent includes utilities for managing cleanup tasks essential for maintaining system integrity. For instance, `HazelcastCleaner.java` manages cleanup operations related to Hazelcast resources, ensuring efficient memory usage and resource deallocation.
- **initialize:** Classes in this section are responsible for system and user interface initialization. `Initializer.java` handles the core setup and configuration of the system, while `InitializeUI.java` focuses on preparing the user interface components for interaction.
- **load:** This module focuses on loading and integrating data into the system. `BooksDatalakeLoader.java` is used to load books into the data lake for further processing, and `HazelcastBookLoader.java` manages the integration of book data with Hazelcast, facilitating distributed data management.
- **services:** The services subcomponent provides the core functionality to manage various operations:
  - `CrawlService.java`: Handles web crawling tasks to gather and process data.
  - `CreateIndexService.java`: Manages the creation of inverted indices for efficient search operations.
  - `CreateMetadataService.java`: Generates metadata to enhance search and data classification.
  - `SearchService.java`: Facilitates search queries across the system using the constructed indices.
- **tasks:** This section is dedicated to managing task execution and prioritization. `PriorityTask.java` assigns priorities to different tasks based on system requirements, while `TaskExecutor.java` handles the execution of these tasks, ensuring smooth operation.
- **update:** Responsible for updating processes within the system. `DatamartUpdater.java` ensures that data marts remain current and synchronized with the latest information.
- **utils:** A collection of utility classes supporting a range of operations. Key examples include:
  - `DataTypeSelector.java`: Assists in selecting appropriate data types during processing.
  - `InvertedIndexUnifier.java` and `MetadataUnifier.java`: Facilitate the unification of indices and metadata for consistent data management.
  - `StopwordsLoader.java`: Loads and manages stopwords to enhance search efficiency and accuracy.

3.2.3 Config:

The **Config** package centralizes the system's essential configurations, ensuring efficient operation and the correct integration of all components. This package allows centralized management of network settings, storage, and configurations related to external services.

- **HazelcastConfig:** The HazelcastConfig file configures a Hazelcast instance as a distributed in-memory storage platform, providing a structure to efficiently handle data in high-concurrency environments. The configuration begins by creating a Config instance, which defines the necessary settings to customize Hazelcast's behavior. One of the initial parameters configured is the cluster name, retrieved from the HZ_CLUSTERNAME environment variable. If this variable is not defined or is empty, a default value of "default-cluster" is assigned. This ensures that Hazelcast instances sharing the same cluster name can communicate with each other.

  Regarding the network configuration, Hazelcast is set to listen on port 5701, with automatic port increment enabled if the default port is already in use. This allows multiple Hazelcast instances to run on the same host without port conflicts. Additionally, multicast discovery is disabled, meaning that nodes will not automatically communicate over the local network. Instead, TCP/IP discovery is used, which can be enabled if cluster members are specified via the HZ_NETWORK_JOIN_TCPIP_MEMBERS environment variable. This variable allows the definition of a comma-separated list of the IP addresses of the nodes that form part of the cluster. If this variable is not defined, TCP/IP discovery is also disabled, ensuring that only explicitly configured nodes are used.

  Another key configuration is the definition of a public address through an environment variable. This is particularly useful in environments where nodes are behind NAT or in cloud infrastructures, enabling nodes to be accessible from outside the local network. Additionally, Hazelcast allows the management of distributed maps, and three specific maps are configured in this file: datalake, invertedIndex, and metadata. Each of these maps has a backup level configured with setBackupCount(1), ensuring that every stored data point has an additional copy on another node in the cluster. This significantly improves the system's resilience, allowing data recovery in case of node failures.

  The use of TCP/IP as the underlying protocol ensures reliable communication between cluster nodes. TCP/IP is a widely used standard for data transmission in networks, and its implementation in Hazelcast allows direct connections between the defined nodes, avoiding reliance on automatic methods like multicast. This approach offers greater control over the cluster topology and enhances security since only authorized nodes can join the system.

3.2.4 Domain:

The **Domain** package establishes the fundamental entities and contracts that define the system's core logic. It provides a structured foundation for implementing functionalities, ensuring a modular, cohesive, and decoupled design.

- **CrawlerService:** This interface defines a contract for crawling services within the system. It specifies a single method that allows extracting a list of books from an external source, starting from an initial identifier and retrieving a defined number of books. Its minimalist design facilitates the implementation of specific adapters for different data sources, promoting extensibility and decoupling.
- **IndexRepository:** This interface defines a contract for managing the inverted index within the system. It provides methods to save and retrieve the inverted index, ensuring its availability for query and analysis operations. Its design allows for the implementation of different storage strategies, promoting flexibility in index management.
- **MetadataRepository:** This interface defines the contract for metadata management within the system. It includes methods to save new metadata and retrieve a complete list of stored metadata, providing a foundation for efficiently handling information associated with documents. Its design allows for the implementation of different storage strategies, ensuring flexibility and scalability in data management.

- **Query:** This interface defines a contract for executing queries within the system. Its single method, `execute()`, allows for the implementation of different types of queries, returning a result encapsulated in a `QueryResult` object. This design facilitates extensibility, allowing new query types to be added without modifying the existing logic.
- **QueryService:** This interface defines a contract for implementing advanced search services in the system. It includes methods for performing different types of queries, such as exact searches, fuzzy searches, and metadata-based searches, interacting with index and metadata repositories as well as Hazelcast instances for distributed in-memory storage. Its design efficiently handles complex queries, facilitating extensibility and decoupling in the implementation of search services.
- **InvertedIndex:** This class defines the structure of the inverted index, enabling the association of keywords with the documents where they appear and their specific positions within those documents. It efficiently manages the storage and retrieval of information through a hierarchical map that organizes words, book identifiers, and data related to word frequency and positions. The class also provides methods to add new words, retrieve existing information, and modify the index as needed, ensuring scalable and structured management of indexed data.
- **Metadata:** This class represents the metadata associated with a book in the system, including information such as the book's identifier, author, publication date, and language. It provides a clear and accessible structure for storing and managing this data, enabling its use in advanced queries and filtering. Additionally, its design includes mechanisms for serialization, facilitating storage and transfer in various system contexts while ensuring flexibility and compatibility.
- **QueryResult:** This class encapsulates the results of a query performed in the system, providing structured information about the books found, the total number of results, and the type of query that generated these results. Additionally, it includes a method to serialize the data into JSON format using the Gson library, facilitating visualization and transfer. It also defines an internal class, `BookInfo`, which stores specific details about a book, such as its identifier, word positions, and frequency, enabling a more detailed analysis of the query results.
- **Book:** This class represents a document within the system, encapsulating two main attributes: a unique identifier and the book's content. The class provides methods to access these attributes via the getters `getId()` and `getContent()`. Its design is simple and focused on providing a basic representation of a book within the system.

3.2.5 Main:

The **Main** class serves as the primary entry point of the system, responsible for coordinating and configuring its essential components, including the user interface (UI) and internal processes. It uses Spark to set up a web server on port 4567 and defines the location of the static files required for users to access the UI through a browser.

A key feature of the class is the dynamic selection of the data type via the DataTypeSelector component. This allows the system to define and adjust the storage format used, providing flexibility to meet different requirements.

The initialization process begins with the Initializer, which configures fundamental services such as indexing, metadata management, and integration with Hazelcast. Once initialized, the CrawlService is activated to collect and structure data, starting with a defined crawling depth.

The system also employs the DatamartUpdater component, which synchronizes the collected data with the existing indexes and metadata, ensuring that the information remains up-to-date and ready for queries. Finally, the class integrates the HazelcastCleaner component, which optimizes distributed memory by removing unnecessary data and periodically synchronizing it with disk-based repositories to maintain operational efficiency.

In summary, the Main class not only configures the server and UI but also enables flexibility in data type selection, coordinates data collection and synchronization, and ensures efficient resource management in distributed systems.

3.2.6 Utils:

The **Utils** package contains utility classes that provide auxiliary functionalities essential for the system's operation. These classes complement the main logic by managing data, cleaning resources, unifying indexes and metadata, and loading stopwords.

- **DataTypeSelector:** This class manages the selection of the storage type for the system's indexes and metadata, based on the specified configuration. It provides an abstraction for creating repositories based on different storage formats, such as plain text, MapDB, or CBOR. This allows the system to be flexible and adaptable to various storage requirements without modifying the core code.
- **HazelcastCleaner:** This class implements a cleaning mechanism for data stored in distributed memory using Hazelcast. It provides functionality to save books to the file system from Hazelcast, persist inverted indexes to local storage, and save metadata to dedicated repositories. Additionally, it ensures that these resources are cleaned from memory to optimize resource usage. The cleaning process is executed periodically and is designed to maintain system efficiency while minimizing the performance impact during high-concurrency operations.
- **InvertedIndexUnifier:** This class unifies the inverted indexes stored both in Hazelcast and in persistent storage. It efficiently merges duplicate data by summing frequencies and positions of words in the corresponding documents. It provides a unified and consolidated index that can be used for subsequent queries.
- **MetadataUnifier:** This class enables the unification of metadata stored in Hazelcast and in persistent repositories. By combining this metadata into a single, duplicate-free list, it ensures that the information associated with the documents is complete and up to date, optimizing its use in advanced queries.
- **StopwordsLoader:** This class loads a set of irrelevant words, or stopwords, from a resource file. These words are used to filter out common terms that do not provide semantic value to the queries. It provides a set of stopwords accessible to different components of the system, ensuring efficient and clean text processing.

3.3. API (Application Programming Interface):

The system's API has been configured using the Spark framework, allowing for a lightweight and flexible implementation of endpoints to enable communication between the client and server. The API is designed to handle HTTP requests through specific routes that process and return results in JSON format. A key aspect of the configuration is the management of CORS (Cross-Origin Resource Sharing), ensuring that external applications can interact with the API securely, even if hosted on different domains. This is achieved by defining headers like Access-Control-Allow-Origin and configuring methods to accept requests from any origin. Additionally, Spark simplifies the management of REST routes, such as the test endpoint /api/test, which returns a simple JSON message to validate the API's status and availability. This approach ensures a reliable programming interface to connect the system's logic with external clients.

In terms of its architecture, the API is integrated with the system's essential services, such as data crawling, index and metadata updates, and resource cleaning in Hazelcast. From the moment the server starts, the configurations defined in the main file (`Main.java`) establish port 4567 to listen for incoming requests and allow access to the static resources required for the user interface. Furthermore, the endpoints are designed to interact directly with the system's internal components, such as Hazelcast and persistent repositories, ensuring efficient and accurate responses. This modular design not only guarantees the flexibility and scalability of the API but also facilitates future extensions or modifications to the offered services.

3.4. UI (User Interface):

The user interface (UI) of the system is configured in the resources/static folder and consists of several essential files: `index.html`, `styles.css` and `app.js`. This structure provides an interactive and visually appealing experience for users interacting with the system. The `index.html` file serves as the core of the interface, defining the main structure of the page and linking to the other files to handle design and

functionality. On the other hand, `styles.css` ensures a consistent appearance by applying styling rules that define colors, fonts, and the layout of elements on the screen. The `app.js` file manages the logic for interacting with the backend, enabling the interface to send requests to API endpoints and dynamically update content based on the received responses.

The server is configured using the Spark Framework to efficiently serve these static resources, ensuring they are available to users when accessing the application. This is achieved with the line staticFiles.location("/static") in the main file `Main.java`, which directs the server to locate these resources. The integration of CORS (Cross-Origin Resource Sharing) allows the interface to communicate with the API, even from different domains, ensuring interoperability. Overall, the user interface is designed to be responsive, adapting to different devices such as computers, tablets, and mobile phones. Its modular configuration allows for easy extension and maintenance, ensuring that the system can evolve and adapt to new requirements efficiently.

4. Experiments and Tests

The experiments and tests conducted in this project are crucial for evaluating the performance and robustness of the implemented system. This section is divided into two main parts: client tests and benchmarking tests. Both aim to analyze different aspects of the search engine's performance, covering its ability to handle concurrent queries and its efficiency in performing critical operations such as writing inverted indexes and metadata.

The client tests were designed to simulate real-world usage scenarios, evaluating how the system responds under various levels of concurrent load. Metrics such as average response time, the number of requests processed, and the number of exceptions generated were analyzed as the number of concurrent threads increased. These tests help identify potential bottlenecks, validate the system's scalability, and assess its stability under high-demand conditions.

On the other hand, the benchmarking tests focused on measuring the performance of the different data structures and storage techniques used in the system. Experiments were conducted to compare the time required to perform key operations, such as writing inverted indexes and metadata, as well as query response times. These tests not only compared the performance of multiple implemented options (such as Binary, Avro, Kryo, CBOR, Hazelcast, and File) but also provided valuable insights into the strengths and limitations of each under different dataset sizes and workloads.

Together, the experiments and tests documented in this section provide a comprehensive view of the system's behavior, highlighting the improvements achieved in each iteration of the project and demonstrating how the applied optimizations contribute to a more efficient and scalable search engine. These results not only support the technical decisions made during development but also establish a solid foundation for future enhancements to the system's architecture and performance.

Client Tests

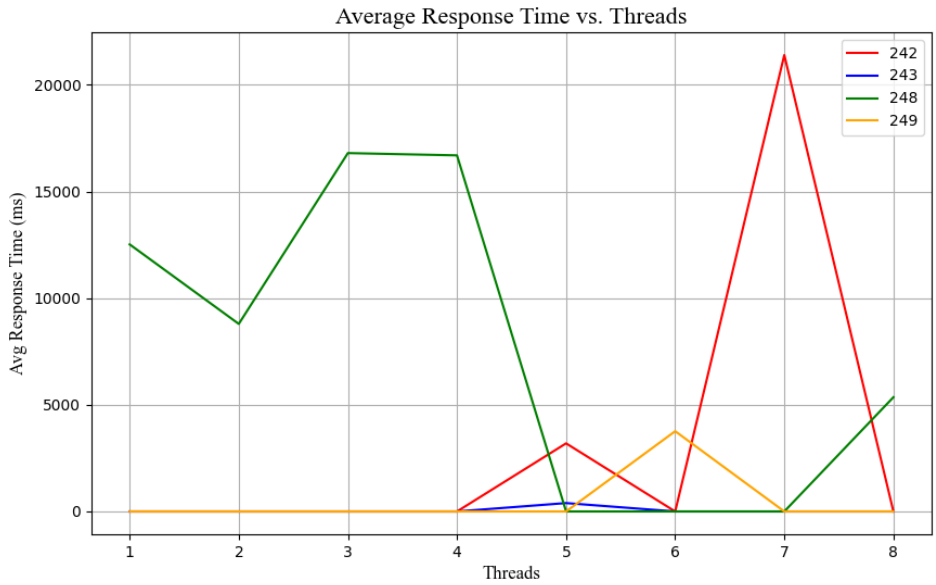

Figure 1: Average Response Time vs. Threads

The graph illustrates the relationship between the number of clients, with multiple threads, and the average response time (in ms) for different nodes. The clients are identified by the last three digits of their IP addresses (242, 243, 248, and 249). Based on the data presented in the graph, the following conclusions can be drawn:

1. **Variability in performance across nodes:**
   - Computer 248 shows significantly higher response times compared to the others, peaking around 16,000 ms with 4 threads. This indicates anomalous behavior or overload on this node.
   - Computers 242, 243, and 249 demonstrate considerably lower response times in most thread configurations, with peaks at different thread counts.
2. **Performance degradation with increased threads:**
   - The average response time tends to increase or become inconsistent as the number of threads rises, suggesting the system struggles to handle higher concurrency levels.
3. **Computer-specific behavior:**
   - Computer 242 exhibits a notable spike at 7 threads, reaching over 22,000 ms before dropping dramatically.
   - Computer 243 shows more stable behavior, maintaining consistently low response times, indicating better tolerance to an increase in thread count.
   - Computer 249 has very low response times in most configurations, except for a slight increase at 6 and 7 threads, where it surpasses the other nodes but remains relatively low overall.
4. **Limited scalability:**
   - The graph highlights a significant bottleneck, as increasing the number of threads leads to overall performance degradation for some nodes (such as 248 and 242). This could be due to resource limitations or lack of optimization.
5. **Response time anomalies:**

- Computer 248 exhibits particularly erratic behavior, with much higher response times than the rest, indicating overload or a specific issue with this node.
- Computer 242 experiences an unexpectedly high spike at 7 threads, suggesting transient performance issues or inefficiency in concurrency management.

These observations underscore the importance of optimizing node configurations and potentially implementing horizontal scaling to enhance the system's ability to efficiently handle concurrency.

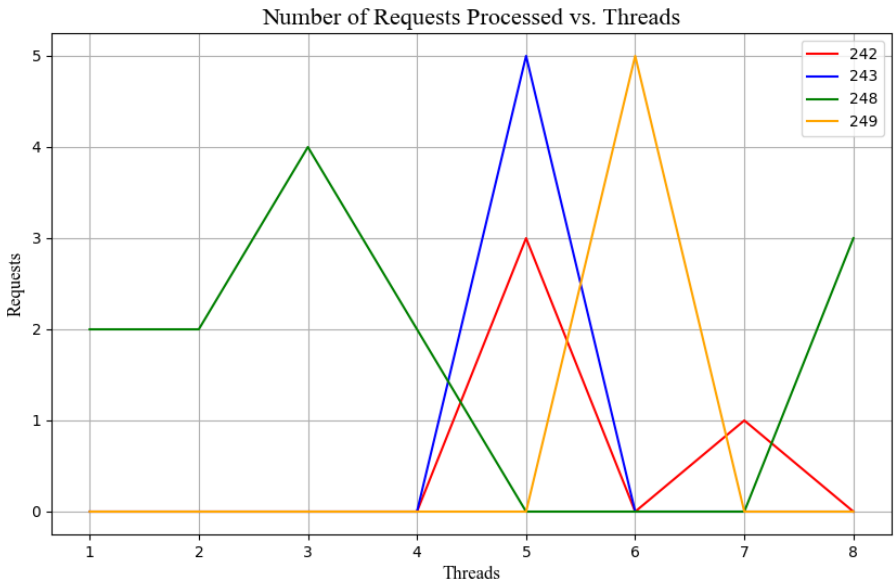

Figure 2: Number of Requests Processed vs. Threads

The graph presents the relationship between the number of clients (242, 243, 248, 249), with multiple threads, and the requests processed across different nodes. The following conclusions can be drawn:

1. **Client-specific performance variability:**
   - Computer 248 demonstrates higher request handling capacity with a peak of 4 requests at 3 threads, followed by a sharp decline.
   - Nodes 242, 243, and 249 show peaks at different thread counts, indicating variability in performance across computers.
2. **Performance degradation beyond specific thread counts:**
   - For all computers, the number of requests drops to zero as the number of threads increases beyond a certain limit (5 threads in most cases).
   - This confirms that the system cannot sustain higher concurrency levels.
3. **Optimal thread counts for request handling:**
   - Computer 248 achieves its best performance at 3 threads.
   - Computer 243 peaks at 5 threads with 5 requests, showing a higher tolerance to concurrency compared to other nodes.
   - Computers 242 and 249 show peaks at 5 and 6 threads, respectively, before their performance deteriorates.
4. **Limited scalability:**
   - The graph highlights a clear bottleneck in scalability, as increasing the number of threads consistently leads to reduced request handling for all nodes.

- This indicates the need for horizontal scaling or system optimization.

5. **Computer 249 as an anomaly:**
   - Computer 249 shows a unique behavior, handling requests better at higher thread counts (6 and 7 threads), but its performance also deteriorates sharply afterwards.

These observations emphasize the need to identify and address node-specific bottlenecks and consider horizontal scaling to improve overall system scalability.

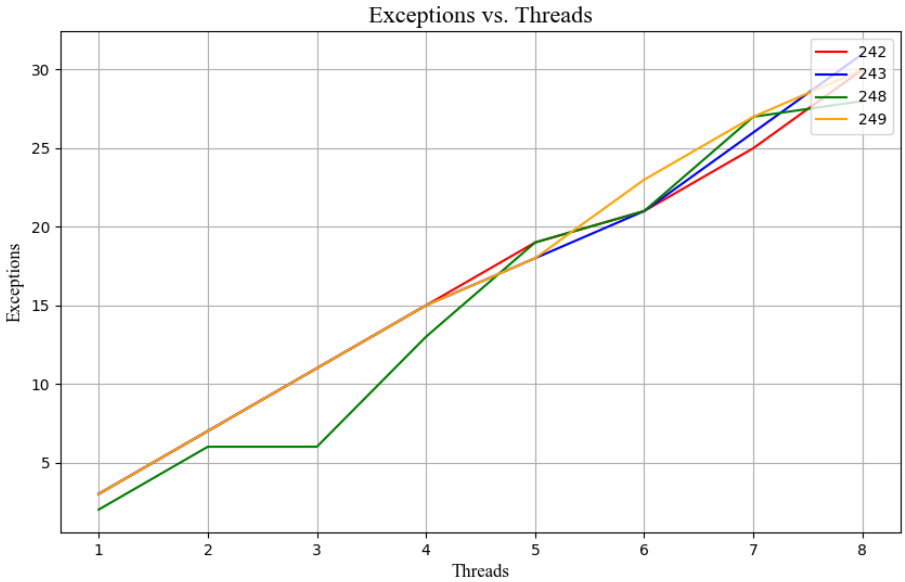

Figure 3: Exceptions vs. Threads

The graph illustrates the relationship between the number of clients (242, 243, 248, 249), with multiple threads, and the exceptions encountered across different nodes. The following conclusions can be drawn:

1. **Linear growth in exceptions:**
   - Across all clients, exceptions increase nearly linearly as the number of threads rises.
   - This trend highlights the system's inability to handle increasing concurrency efficiently.
2. **Client-specific behavior:**
   - Client 248 exhibits a slower rise in exceptions at lower thread counts (up to 3 threads), indicating better performance under limited concurrency.
   - Client 242, 243, and 249 show a more consistent linear increase, with minimal differences in exception rates at higher thread counts (6-8 threads).
3. **Convergence at high thread counts:**
   - All nodes converge to similar exception rates at 7-8 threads, ranging between 30 and 32 exceptions.
   - This suggests that the bottleneck is systemic rather than node-specific when the system is under high load.
4. **Implications for scalability:**
   - The consistent rise in exceptions indicates that the system is not scalable in its current configuration and struggles to maintain stability under high concurrency.
   - Scaling horizontally by adding more nodes or optimizing the current architecture is necessary to handle increased loads.

These findings reinforce the need for architectural improvements to mitigate exception rates and improve overall system performance.

Benchmark Tests

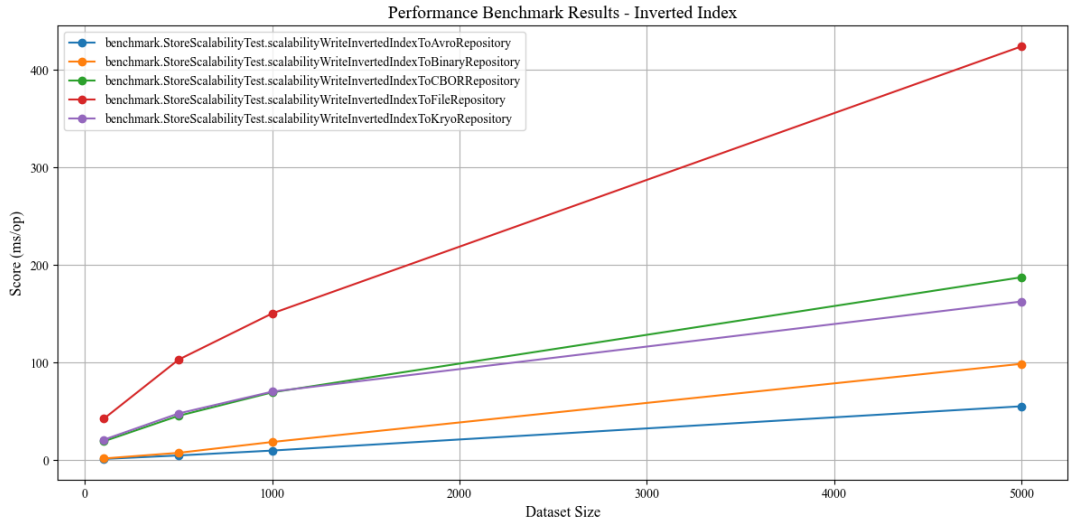

Figure 4: Performance Benchmark Results - Inverted Index

The figure 4 shows the performance results for writing the inverted index using different data repositories: Avro, Binary, CBOR, File, and Kryo. The horizontal axis represents the dataset size, ranging from 0 to 5000, while the vertical axis measures the score in milliseconds per operation (ms/op). This analysis allows us to evaluate how dataset size impacts performance when writing the inverted index to each type of repository and to compare the efficiency of the different implementations.

The Avro repository, represented by the blue line, shows a linear growth in score as the dataset size increases. It maintains a relatively consistent and low performance compared to other repositories like CBOR or File. This indicates that Avro is suitable for handling larger datasets with moderate penalties in write times, likely due to its optimized design for efficient serialization.

The Binary repository, represented by the orange line, has one of the best performances. Its curve remains low across various dataset sizes, suggesting that this format is highly efficient for writing the inverted index. This may be due to its simplicity and lack of overhead associated with more complex methods. Binary is an excellent option for applications where performance is critical, and the data structure does not require advanced features.

The Kryo repository, represented by the purple line, shows a moderately growing curve. This behavior highlights that Kryo offers a good combination of efficiency and flexibility, making it especially useful for applications requiring advanced serialization with acceptable performance. However, it does not reach the efficiency of Binary or Avro, which could be a disadvantage in scenarios where write times are critical.

The CBOR repository, represented by the green line, exhibits intermediate performance. Its curve grows moderately as the dataset size increases, indicating that CBOR provides reasonable efficiency for writing the inverted index, although it is not as optimized as Binary or Avro. CBOR can be a viable option when format compactness is prioritized, making it a useful alternative in applications where storage space is a key consideration.

The File repository, represented by the red line, has the worst performance of all. Its curve grows sharply as the dataset size increases, suggesting that File introduces significant overhead when writing data. This makes it less suitable for handling inverted indexes with large datasets. While File may be useful in scenarios where implementation simplicity is a priority, its poor performance makes it a limited option in this use case.

Comparing the different repositories, Avro stands out as the most efficient, making it ideal for applications where performance is a priority, especially with large datasets. Avro not only demonstrates

outstanding performance in write times but also provides efficient serialization capabilities, making it suitable for scenarios requiring flexibility and structured data management. Binary, while efficient, occupies a slightly lower position than Avro in terms of overall performance. Kryo and CBOR exhibit intermediate performance, with reasonable write times but not as optimized as Avro or Binary. CBOR can be an interesting option when data compactness is important, but overall, these options are not as recommendable as Avro for maximizing efficiency. File, on the other hand, is clearly the worst option due to its significant penalty in write times, making it ineffective for inverted indexes with large datasets.

In conclusion, the choice of repository depends on the specific needs of the system. To maximize performance, Avro is the ideal option. Avro not only offers the best performance in terms of write times but also combines efficiency and advanced serialization capabilities, making it perfect for handling inverted indexes in large datasets. If a balanced alternative between simplicity and functionality is needed, Binary is a viable option, although slightly less efficient than Avro. On the other hand, File should be avoided, as it performs the worst, showing significant penalties in write times, making it unsuitable for large volumes of data. CBOR, while better than File, is also not an optimal choice due to its inferior performance compared to Avro and Binary. These results highlight the importance of carefully selecting the data format based on the project's requirements, ensuring optimization in processing times, scalability, and efficient resource management.

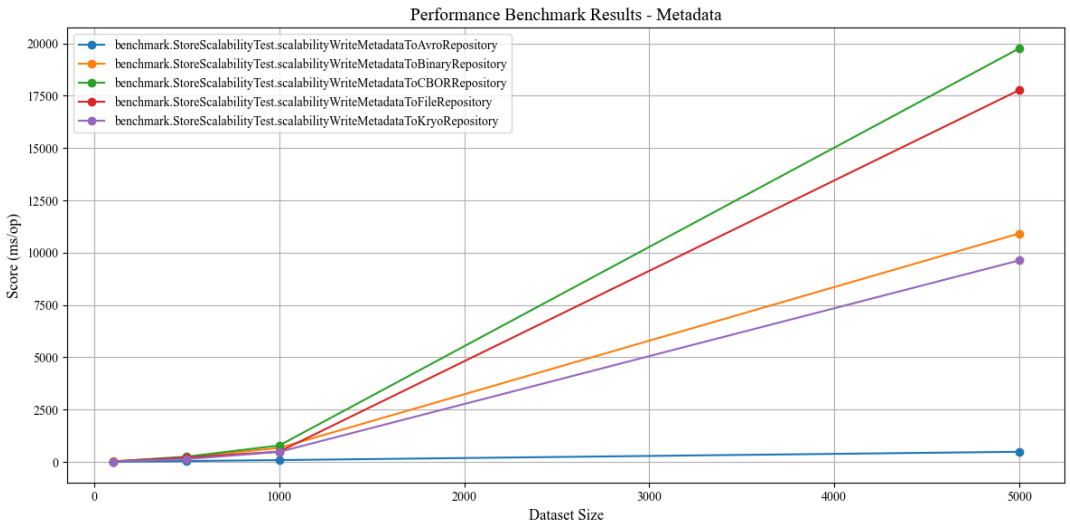

Figure 5: Performance Benchmark Results - Metadata

The figure 5 shows the performance results for writing metadata using different data repositories: Avro, Binary, CBOR, File, and Kryo. The horizontal axis represents the dataset size, ranging from 0 to 5000, while the vertical axis measures the score in milliseconds per operation (ms/op). This analysis evaluates how dataset size impacts performance when writing metadata to each type of repository and compares the efficiency of the different implementations.

The Avro repository, represented by the blue line, is clearly the most efficient. Its curve remains flat across all dataset sizes, indicating that it offers exceptionally consistent performance, even with large volumes of data. This behavior demonstrates that Avro is optimized for handling structured data efficiently, making it the best option for writing metadata.

The Kryo repository, represented by the purple line, ranks second in performance. Its curve shows moderate linear growth as the dataset size increases, making it significantly more efficient than options like File or CBOR, although not as optimal as Avro. Kryo stands out for its ability to handle metadata with good efficiency and flexibility, making it a solid alternative for applications requiring advanced serialization.

The Binary repository, represented by the orange line, takes third place. Its performance is reasonable, but its curve has a steeper growth compared to Kryo, indicating that it starts to lose efficiency as the dataset size grows. While Binary remains a viable option, its performance is inferior to Avro and Kryo for metadata writing.

The File repository, represented by the red line, shows mediocre performance. Its curve grows sharply, reflecting a significant penalty in write times as the dataset size increases. This indicates that File is not an efficient option for writing metadata, especially in applications handling large volumes of data.

The CBOR repository, represented by the green line, performs the worst. Its curve grows abruptly and surpasses all other repositories in write time penalties, making it the least suitable option for handling metadata. While CBOR may be useful in scenarios where data compactness is prioritized, its poor performance makes it ineffective for most use cases involving metadata writing.

Compared to the results for the inverted index, Avro maintains its position as the most efficient repository in both cases, demonstrating its versatility and optimization for both structured data and metadata. Kryo also delivers good performance in both scenarios, standing out as a reliable option. Binary, while reasonably efficient in both cases, performs worse than Avro and Kryo. File and CBOR, on the other hand, consistently show poor performance, with File being the worst for inverted indexes and CBOR the worst for metadata.

In conclusion, Avro is the most recommended option for both inverted indexes and metadata due to its outstanding efficiency and consistent performance. Kryo emerges as a solid alternative in both cases, while Binary offers acceptable performance but with limitations as dataset sizes grow. File and CBOR should be avoided in applications where performance is critical, as their poor performance represents a significant disadvantage in terms of scalability and processing time.

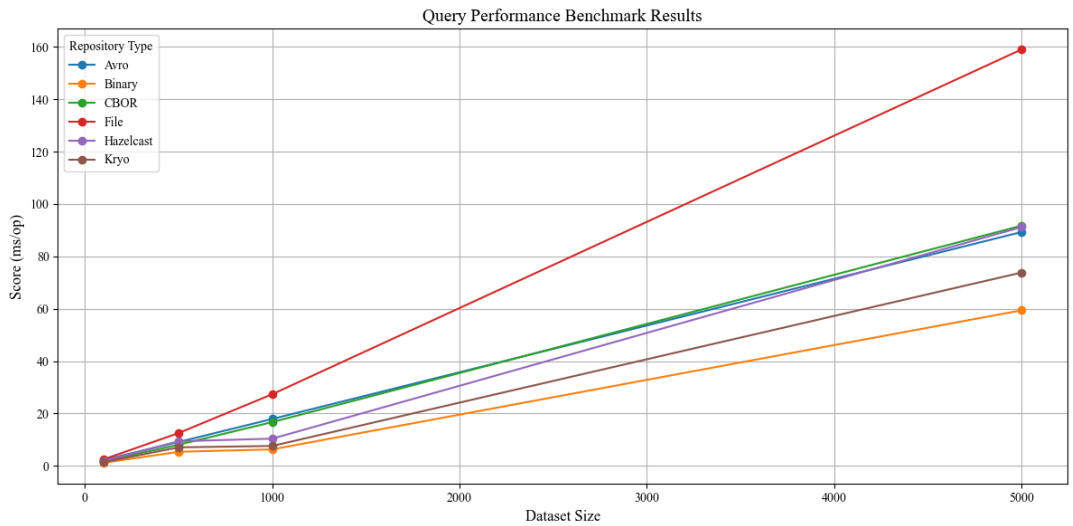

Figure 6: Query Performance Benchmark Results

The figure 6 shows the performance results for queries across different data types: Binary, Kryo, Hazelcast, CBOR, Avro, and File. The horizontal axis represents the dataset size, ranging from 0 to 5000, while the vertical axis measures the score in milliseconds per operation (ms/op). This analysis evaluates how dataset size affects query response times for each type of repository and compares the efficiency of the different implementations.

The Binary repository, represented by the orange line, stands out as the most efficient in this case. Its curve remains the lowest across all dataset sizes, indicating superior performance in queries regardless of the data volume. This behavior demonstrates that Binary is highly efficient for handling queries, likely due to its simplicity and low computational cost.

The Kryo repository, represented by the brown line, takes second place in performance. Its curve is slightly steeper than Binary's but remains competitive, showing excellent performance in scenarios where advanced serialization and query performance are important. Kryo positions itself as a solid alternative for applications requiring a balance between efficiency and flexibility.

The Hazelcast repository, represented by the purple line, ranks third. Its performance is reasonable, with a curve that grows moderately as the dataset size increases. Hazelcast proves to be a reliable option

for queries in distributed systems, especially when decent performance is required alongside shared in-memory storage capabilities.

The CBOR repository, represented by the green line, shows intermediate performance. Its curve is steeper than Hazelcast's, indicating a greater penalty in query response times as the dataset grows. Although CBOR offers advantages in terms of data compactness, its query performance places it below more efficient options like Binary or Kryo.

The Avro repository, represented by the blue line, delivers lower-than-expected performance, occupying the second-to-last position in this evaluation. Its curve shows moderate growth but lags behind options like Binary, Kryo, and Hazelcast in terms of query response times. Despite being an excellent choice for writing data, Avro does not maintain the same level of efficiency in queries.

The File repository, represented by the red line, performs the worst of all in this graph. Its curve grows steeply, significantly surpassing the other repositories in terms of time penalties as the dataset size increases. This indicates that File is not an efficient option for queries, especially in systems handling large data volumes. Its poor performance makes it a limited choice for this type of operation.

Compared to previous results, where Avro was the best option for writing structured data and metadata, this graph shows a significant shift. While Avro excels in write operations, it is not equally efficient for queries, being outperformed by Binary, Kryo, and Hazelcast in this case. File, consistent with its poor performance in other scenarios, once again ranks as the worst option.

In conclusion, Binary is the most recommended option for performing queries due to its outstanding performance and consistent efficiency. Kryo and Hazelcast are also viable options, offering a good balance between performance and advanced features. CBOR and Avro, while reasonable, are not the most optimal choices for intensive queries. Lastly, File should be avoided for this type of operation due to its poor performance, which represents a significant disadvantage in terms of response time and scalability.

5. Performance Comparison of Search Implementations

The performance comparison across the different stages of the project is a key aspect of this study, allowing for a detailed evaluation of the evolution and improvements achieved in previous iterations of the search engine's development. This section focuses on analyzing the performance differences among the three main stages: the initial implementation in Python during the first stage, the transition to Java in the second stage, and the final optimized implementation in Java in the third stage.

Each prior stage introduced new techniques, optimizations, and data structures that progressively enhanced the system's efficiency and scalability. By comparing these implementations, it is possible to identify the impact of transitioning to a more robust programming language, integrating advanced serialization and storage mechanisms, and applying architectural refinements. The analysis covers aspects such as search times, resource usage, and scalability capacity, highlighting the strengths and limitations of each prior stage. This approach provides a clear view of the progress achieved and the key decisions that led the system to significantly improved performance in its final version.

5.1. Stage 1 (Python) and Stage 2 (Java)

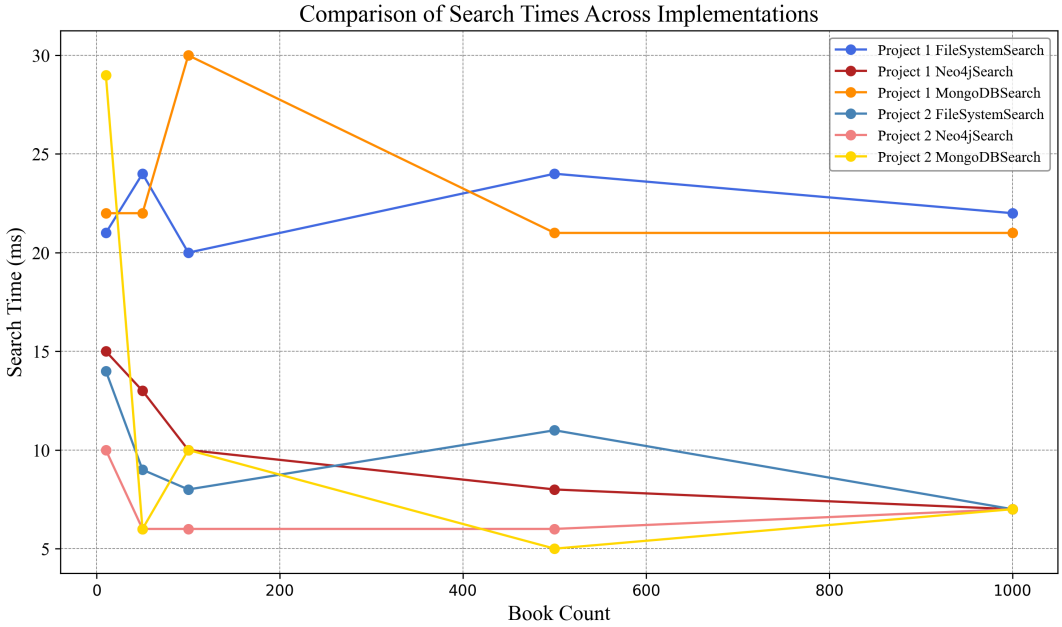

Figure 7: Comparison of Search Times with Varying Number of Books

The figure 7 compares the search times obtained in two different projects: the first implemented in Python and the second in Java. In both projects, various data structures were used to perform the searches: FileSystem, Neo4j, and MongoDB, with the addition of Binary in the second project. The horizontal axis represents the number of books processed, ranging from 0 to 1000, while the vertical axis measures the search time in milliseconds (ms). This analysis is crucial to evaluate how the programming language and data structures influence performance as the number of books scales.

In stage 1 (Python), the search times are generally higher than in stage 2. For example, the blue line, corresponding to the implementation with FileSystemSearch, shows search times ranging from 20 to 25 ms even for small datasets. This suggests that using FileSystem in Python introduces significant performance penalties, likely due to the overhead associated with file handling in this language.

The orange line, representing MongoDBSearch in Python, shows inconsistent performance, with peaks reaching 30 ms for small datasets and a gradual reduction as the number of books increases. Although MongoDB improves with dataset size, the initial high times indicate greater latency in Python, possibly due to initial configuration overhead or communication inefficiencies with the database.

On the other hand, Neo4jSearch in Python, represented by the red line, exhibits better performance than the other data structures in stage 1, with search times ranging from 7 to 15 ms. This demonstrates that Neo4j is more efficiently integrated into Python compared to the other options, although it still lags behind the implementations in Java.

In stage 2 (Java), the results are significantly better across all cases. The light blue line, corresponding to FileSystemSearch, shows much lower and more consistent search times than in stage 1, with values ranging from 7 to 14 ms even for large datasets. This indicates that the Java implementation greatly optimizes interaction with the file system, reducing overhead.

The pink line, representing Neo4jSearch in Java, maintains consistently low search times, staying below 10 ms for all dataset sizes. This highlights Java's advantages in terms of communication and handling graph-oriented databases, eliminating the latencies observed in stage 1.

MongoDBSearch in Java, represented by the yellow line, also outperforms its Python equivalent, with search times consistently around 5 to 10 ms for large datasets. This result emphasizes Java's efficiency in handling NoSQL databases like MongoDB, achieving more stable and faster performance.

Overall, the results confirm that stage 2 (Java) delivers superior performance compared to stage 1 (Python) for all the data structures evaluated. The improvements are evident in both stability and lower search times, highlighting Java's inherent advantages in memory management, database communication, and file access optimization.

In conclusion, while stage 1 shows significant limitations due to the overhead associated with Python, stage 2 implemented in Java demonstrates a notable improvement in terms of performance and scalability. This suggests that Java is a more suitable option for applications requiring high efficiency in searches over large datasets, especially when integrating advanced data structures.

5.2. Stage 2 (Java) and Stage 3 (Java)

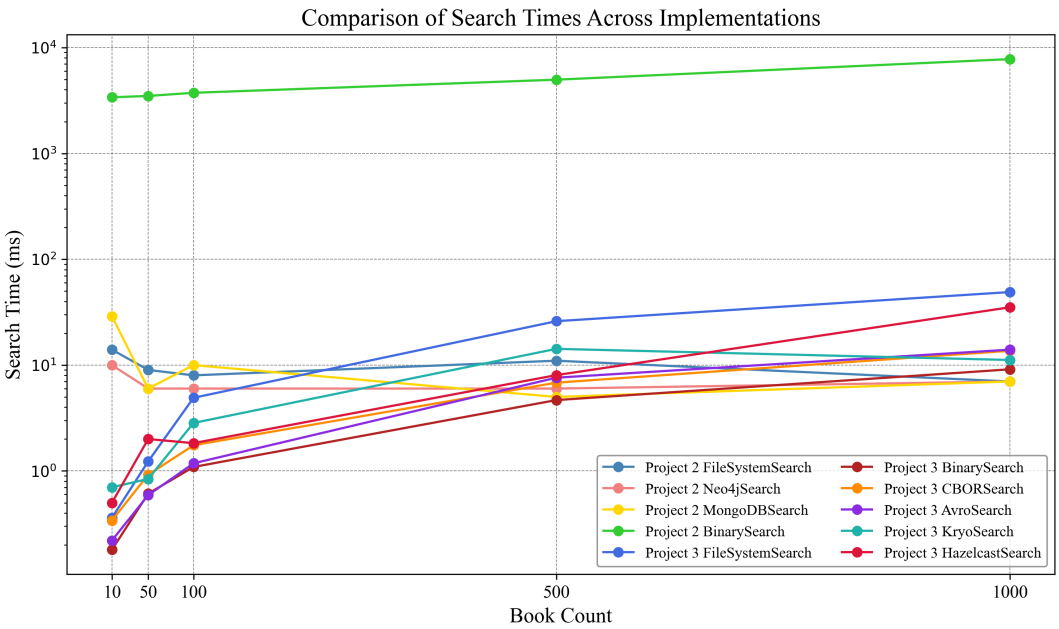

Figure 8: Comparison of Search Times with Varying Number of Books

The figure 8 compares the search times obtained in stage 2 (Java) and stage 3 (Java). In both cases, multiple data structures are evaluated, with a broader variety introduced in the third project. The horizontal axis represents the number of books processed, ranging from 10 to 1000, while the vertical axis, displayed on a logarithmic scale, measures search time in milliseconds (ms). This analysis highlights how the optimizations and new data structures implemented in the third project impact performance as the dataset size scales.

Stage 3 (Java) demonstrates clear improvements compared to stage 2, with BinarySearch being the most efficient option. Represented by the strong red line in the third project, Binary consistently achieves the lowest search times across the entire graph, with values around 10 ms for all dataset sizes. This underscores Binary's exceptional ability to handle queries, optimizing both time and resource utilization.

The purple line in the third project, corresponding to AvroSearch, ranks second in performance. Its search times are consistently low and close to those of Binary, showing considerable improvement compared to the data structures in stage 2. Avro proves to be a solid choice for handling large volumes of data with efficient performance.

CBORSearch, represented by the orange line, ranks third in stage 3. While its search times are slightly higher than Binary and Avro, CBOR still demonstrates competitive performance, making it a viable option in scenarios where data compactness is a priority.

The light red line in the third project, corresponding to HazelcastSearch, takes fourth place. Although less efficient than Binary, Avro, and CBOR, Hazelcast maintains reasonable search times, showcasing its strength as a distributed solution capable of handling large data loads with an acceptable time penalty.

KryoSearch, represented by the light blue line in the third project, has intermediate performance. Its search times are consistent, though not as low as the aforementioned options. Kryo remains a reliable choice for applications requiring advanced serialization features.

In contrast, the data structures in stage 2 (Java), while better than those in the earlier Python stage, are less efficient compared to those in stage 3. Neo4jSearch (pink line in stage 2) is the fastest among stage 2 implementations, maintaining search times below 100 ms for most dataset sizes. However, its performance is overshadowed by the options in stage 3.

FileSystemSearch in stage 2, represented by the light blue line, shows moderate search times but is surpassed by the optimized implementations in stage 3. On the other hand, MongoDBSearch (yellow line in stage 2) exhibits significantly higher search times, especially for larger datasets, indicating limitations in its integration and query handling.

Finally, BinarySearch in stage 2 (green line in stage 2) has the worst performance among all options, with extremely high search times. This highlights the lack of optimization in Binary's implementation during the second stage, in stark contrast to its outstanding performance in the third project.

In summary, stage 3 (Java) demonstrates substantial improvement over stage 2, thanks to the inclusion of new data structures and optimizations. Binary and Avro stand out as the fastest and most efficient options, followed by CBOR and Hazelcast, which provide solid performance. In contrast, the data structures in stage 2, while reasonable, show clear limitations compared to the innovations introduced in stage 3. This underscores the importance of selecting appropriate data structures and applying specific optimizations to maximize system efficiency.

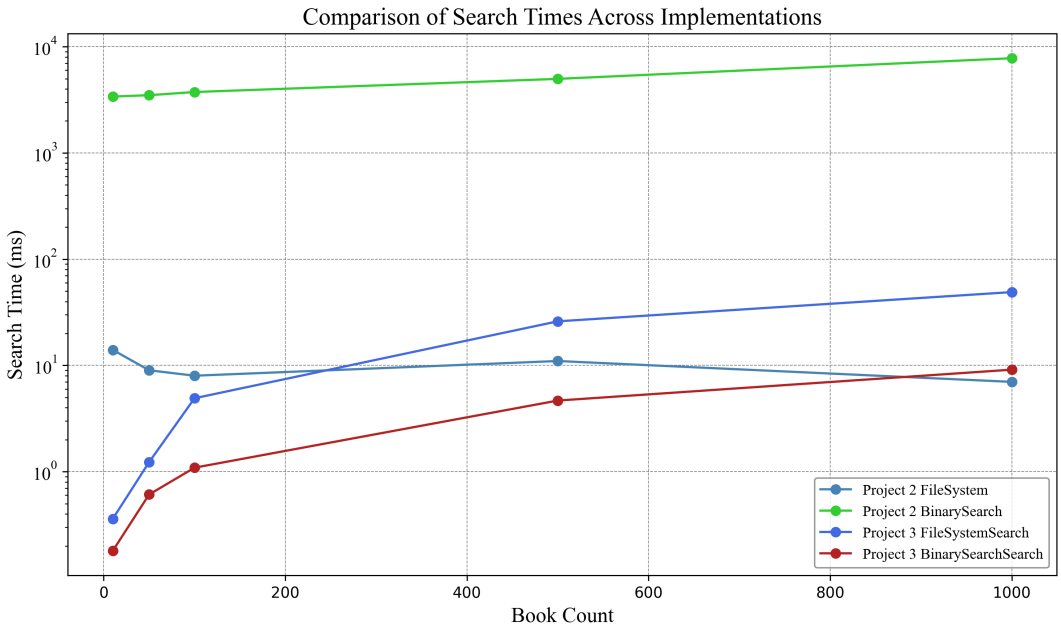

Figure 9: Comparison of Search Times with Varying Number of Books (Binary and File System)

The figure 9 compares search times between stage 2 and stage 3, both implemented in Java, using only the data types that are common to both projects: FileSystem and Binary. The horizontal axis represents the number of books processed, ranging from 0 to 1000, while the vertical axis, on a logarithmic scale, measures search time in milliseconds (ms). This analysis provides a direct comparison of the impact of the optimizations introduced in the third project on these specific data structures.

The BinarySearch from stage 3, represented by the red line, is clearly the most efficient of the four. Its search times remain consistently low, around 10 ms for all dataset sizes. This highlights the effectiveness of the optimizations made in the third project, allowing BinarySearch to handle queries exceptionally well, making it the best option in this comparison.

The FileSystemSearch from stage 3, represented by the dark blue line, ranks second in performance.

Although its search times are slightly higher than those of BinarySearch from stage 3, they remain significantly lower and more consistent than the corresponding implementations in stage 2. This demonstrates that the improvements in integration and file system handling in the third project resulted in a more efficient performance.

The FileSystemSearch from stage 2, represented by the light blue line, shows acceptable performance, although inferior to the FileSystem from the third project. Its search times are higher and display greater variability as the dataset size increases, indicating that the implementation in the second project is not as optimized as in the third.

Finally, the BinarySearch from stage 2, represented by the green line, performs the worst of the four. Its search times are significantly higher than those of the other structures, exceeding 10,000 ms for larger datasets. This highlights the clear limitations of the Binary implementation in the second project, contrasting sharply with the outstanding performance of the same data type in the third project.

In summary, the results demonstrate that the optimizations made in stage 3 had a significant impact on the performance of the common data structures. BinarySearch from stage 3 is the most efficient option, followed by FileSystemSearch from the same project. The implementations in stage 2, while functional, are clearly inferior, with BinarySearch from stage 2 being the least efficient of all. This analysis underscores the importance of improvements in design and implementation to maximize efficiency in systems handling large volumes of data.

6. System Performance Analysis

6.1. How many concurrent clients can the system handle?

The scalability analysis of the system reveals significant limitations in concurrent client support.

Exception Trends: Across all nodes (242, 249, 248, 243), as the number of threads increases, exceptions rise rapidly beyond a certain point. This indicates that the system cannot handle concurrency beyond a specific threshold without experiencing performance degradation.

Processed Requests: The number of processed requests varies by node, but in most cases, performance declines sharply after 5 or 6 threads. For example:

- In nodes like 242 and 249, high exception rates and elevated response times occur when the number of threads exceeds 5.
- In node 248, saturation begins at 4 threads, where response times average 16,692 ms, and exceptions rise to 13. Beyond this point, exceptions exceed 27, and processed requests drop to near zero.

Conclusion: The system is capable of concurrently handling between 4 and 5 clients per node before performance degrades. This corresponds to approximately **20 concurrent clients** across the entire set of 5 nodes analyzed.

6.2. Where are the bottlenecks identified?

Observed Bottlenecks: Load testing highlights the following bottlenecks:

1. **Exceptions growth with increasing threads:** Bottlenecks emerge clearly when threads exceed 4-5 per node. Beyond this threshold, exceptions grow almost linearly, and the system ceases to process requests effectively.
2. **High response times:** Starting from 3-4 threads, average response times exceed 16 seconds in node 248, which is unacceptable for most applications. Similar trends are observed across other nodes.
3. **System collapse under high concurrency:** Even with reduced `sleep_time`, the system does not recover significantly. In nodes like 243 and 242, response times remain at zero while exceptions continue to increase.

Conclusion: These tests are expected to confirm the per-node concurrency limits and provide insights into overall system behavior under combined loads. Improvements can include:

- Load redistribution across additional nodes.
- Query optimization or adjustments to the load balancer configuration.

6.3. Key Findings for Scalability

- The system supports approximately **20 concurrent clients** before experiencing significant performance degradation.
- Load tests identify the main bottlenecks as per-node concurrency limits, with saturation occurring at around **4-5 threads per node**.

These findings emphasize the need for horizontal scaling or enhancements to individual node capacity to achieve better scalability.

7. Conclusion

The conclusion of this project highlights the progressive evolution and improvement in the design and implementation of a search engine through three main stages, each marked by significant advances in technology and architecture. From the initial foundations in Python to the more advanced versions in Java, this work demonstrates a consistent focus on scalability, efficiency, and robustness in handling large volumes of data.

In the first phase, the project established the conceptual foundation by implementing a basic search engine using Python. This stage was crucial for understanding the interactions between fundamental components, such as the crawler, the indexer, and the query engine, serving as a starting point for future enhancements. However, the inherent limitations of Python, particularly in memory handling and concurrency, restricted the system's ability to scale and efficiently respond to complex queries. These limitations were evident in the relatively high response times and instability under concurrent loads.

The transition to Java in the second phase marked a significant advance, leveraging the capabilities of this language to handle concurrent queries and large data volumes. During this stage, data structures such as the inverted index were optimized, and new options were introduced, including integration with databases like Neo4j and MongoDB. The results showed clear improvements in performance and stability, particularly compared to the Python implementation. However, there were still areas for improvement, particularly regarding system scalability and the inclusion of advanced load-balancing techniques.

The third phase represented the culmination of this project, elevating the search engine to an entirely new level through the incorporation of advanced technologies and architectures. The adoption of a hexagonal architecture ensured a clear separation between the core system logic and technical details, enabling greater flexibility and adaptability. Additionally, the use of tools like Hazelcast and Docker transformed the system into a distributed solution capable of handling large volumes of concurrent queries with optimized performance. Hazelcast, with its distributed in-memory storage capabilities, significantly reduced data access times, while Docker facilitated the scalable deployment of multiple search engine instances.

The third project also benefited from extensive experimentation with different data structures, including Kryo, Avro, CBOR, and Binary, among others. These options not only optimized the handling of indexes and metadata but also proved critical in improving system efficiency in terms of response times and resource consumption. Performance test results confirmed that Binary and Avro are the most efficient options for search and write operations, while other structures like CBOR and Hazelcast offered a suitable balance between flexibility and performance.

Another key aspect was the implementation of an Nginx-based load balancer, which efficiently distributed queries across multiple nodes. This functionality, combined with extensive testing, ensured that the system could handle significant concurrent loads without compromising stability or performance. Experiments conducted with simulations of concurrent clients identified and mitigated potential bottlenecks, ensuring that the search engine could operate reliably even in high-demand scenarios.

Overall, this project not only achieved its initial objectives but also exceeded expectations by creating an advanced system that combines efficiency, scalability, and robustness. The improvements introduced at each stage reflect an iterative and methodical approach to optimization, demonstrating the importance of architectural and technological decisions in the development of complex systems. This work not only lays the foundation for future developments in search engines but also provides a model for addressing similar challenges in the realm of Big Data and modern computing.

In conclusion, the Search Engine project represents an outstanding example of how a combination of technological innovation, advanced architectural design, and a data-driven approach can lead to effective solutions for complex problems. From its beginnings in Python to the most advanced version in Java with modern tools, this project provides a solid foundation for future research and applications in the field of large-scale data management and querying.

8. Future Work

In the future, we plan to expand this project by deploying the system on cloud platforms such as Google Cloud and AWS. This step will be essential to improving the scalability, availability, and responsiveness of the system. By leveraging the advantages of these platforms, such as resource flexibility and automatic load balancing, the search engine will be able to handle more simultaneous users while maintaining strong performance even during high-demand situations.

Additionally, we will integrate advanced tools and services offered by these platforms, such as distributed databases, caching systems, and content delivery networks (CDN). These enhancements will further optimize data access and retrieval, reducing response times and improving the user experience.

Another area of development will focus on optimizing query processing. This includes implementing machine learning algorithms to improve result ranking, real-time trend analysis, and personalized searches based on user behavior. These improvements will not only make the system more efficient but also provide users with more relevant and practical results.

We also plan to explore and integrate new technologies that complement the current system's capabilities. For instance, advanced databases to manage complex data relationships or technologies that enable continuous updates of large volumes of data without impacting performance.

Moreover, we aim to optimize distributed memory and data storage management by exploring alternatives that allow faster and more efficient access to indexes and metadata. This could involve adjustments to current tools or combining them with specialized solutions for specific scenarios.

In terms of system design, we will continue to focus on approaches that enhance modularity and flexibility. For example, adopting a microservices-based architecture where each system functionality operates as an independent service will make maintenance, updates, and the integration of new features easier.

Finally, we aim to broaden the project's scope to adapt it to new uses and contexts. This could include applications such as large-scale data analysis, personalized recommendation engines, or search systems that combine text with visual analysis, like images or videos. These expansions will ensure the system remains relevant and up-to-date with the evolving demands of the technological landscape.

In summary, the future of the project will focus on improving its scalability, efficiency, and flexibility through the use of accessible and practical technologies. This approach will allow the system to evolve and adapt continuously, establishing itself as a reliable and effective tool in the field of search engines and data management.